

Explain the difference between list, tuple, set, and dictionary.
What is a list comprehension ? Give one example.
How can you copy a list? Explain shallow vs deep copy briefly.
Explain try except, and finally block.

.In Python, lists and tuples are ordered collections, but lists are mutable while tuples are immutable. Sets are unordered collections of unique elements, and dictionaries store data as key–value pairs.

What is a List Comprehension?

A list comprehension in Python is a concise way to create lists.

It lets you generate a new list by applying an expression to each item in an iterable (like a list, range, or string), optionally including a condition.

It's essentially a one-liner shortcut for writing loops that build lists.

Ways to Copy a List

1. Assignment (not a copy!)

```
a = [1, 2, 3]
```

```
b = a
```

Here, is just another reference to the same list.

- Changing will also change .

2. Shallow Copy

- Creates a new list object, but elements inside are still references to the original.
- Methods:

```
import copy
```

```
a = [1, 2, 3]
```

```
b = a.copy()      # method
```

```
c = list(a)       # constructor
```

```
d = a[:]          # slicing
```

Works fine for simple lists, but if the list contains nested lists, changes inside the nested list will affect both copies.

3. Deep Copy

- Creates a completely independent copy, including nested objects.

- Example:

```
import copy
```

```
a = [[1, 2], [3, 4]]
```

```
b = copy.deepcopy(a)
```

```
import copy
```

```
# Original list with nested list
```

```
a = [[1, 2], [3, 4]]
```

```
# Shallow copy
b = copy.copy(a)
b[0][0] = 99
print("a:", a) # [[99, 2], [3, 4]]
print("b:", b) # [[99, 2], [3, 4]]
```

```
# Deep copy
c = copy.deepcopy(a)
c[0][0] = 42
print("a:", a) # [[99, 2], [3, 4]]
print("c:", c) # [[42, 2], [3, 4]]
```

♦ Copying a List

There are several ways to copy a list:

- Assignment (not a real copy)

Changing `b` will also change `a`.

- Shallow Copy

Creates a new list object, but elements inside are still references to the original.

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a) # or a.copy(), list(a), a[:]
Now, modifying b won't affect a.
```

```
import copy
```

```
a = [[1, 2], [3, 4]]
```

```
# Shallow copy
b = copy.copy(a)
b[0][0] = 99
print("a:", a) # [[99, 2], [3, 4]]
print("b:", b) # [[99, 2], [3, 4]]
```

```
# Deep copy
c = copy.deepcopy(a)
c[0][0] = 42
print("a:", a) # [[99, 2], [3, 4]]
print("c:", c) # [[42, 2], [3, 4]]
```

```
try:
    # risky code
except SomeError:
    # handle the error
finally:
    # always executed
```

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("You can't divide by zero!")
finally:
    print("Execution finished.")
```

```
try:
    f = open("data.txt", "r")
    content = f.read()
except FileNotFoundError:
    print("File not found.")
finally:
    print("Closing file if it was opened.")
    try:
        f.close()
    except:
        pass
```

```
▶ cubes={x:x**3 for x in range(1,6)}
  print(cubes)
... {1: 1, 2: 8, 3: 27, 4: 64, 5: 125}
```

```
▶ try:
  result=10/0
except ZeroDivisionError:
  print("Error divide by zero")
... Error divide by zero
```

```
▶ words=input("Enter words separated by space:").split()
  filtered =[word for word in words if len(word)<4]
  print("filtered word:", filtered)
... Enter words separated by space:10
  filtered word: ['10']
```
